

# AXI4-Lite Firewall IP Core

## User Guide

|                  |                                               |
|------------------|-----------------------------------------------|
| IP core          | <code>altera_axi4_lite_firewall</code>        |
| Core version     | 2.0                                           |
| Document version | 1.0                                           |
| Date             | August 2026                                   |
| Target           | Intel FPGA / Quartus Prime, Platform Designer |

# Contents

---

|                                                   |           |
|---------------------------------------------------|-----------|
| <b>1. About the AXI4-Lite Firewall IP Core</b>    | <b>4</b>  |
| 1.1 Features                                      | 4         |
| 1.2 Device Family Support                         | 4         |
| 1.3 Resource Utilization                          | 4         |
| 1.4 Release Information                           | 5         |
| <b>2. Getting Started</b>                         | <b>7</b>  |
| 2.1 Installing the IP Core                        | 7         |
| 2.2 Adding the Core to a Platform Designer System | 7         |
| 2.3 Required Connections                          | 8         |
| 2.4 Simulating the IP Core                        | 8         |
| 2.5 Files Provided                                | 9         |
| <b>3. Functional Description</b>                  | <b>10</b> |
| 3.1 Block Diagram                                 | 10        |
| 3.2 Access Control                                | 13        |
| 3.3 Rule Evaluation                               | 14        |
| 3.4 Fault Isolation and Timeout                   | 14        |
| 3.5 Recovery Sequence                             | 16        |
| 3.6 Interrupts                                    | 18        |
| 3.7 Control Port Behaviour                        | 19        |
| 3.8 Reset                                         | 19        |
| 3.9 Latency                                       | 19        |
| <b>4. Parameters</b>                              | <b>20</b> |
| <b>5. Interface Signals</b>                       | <b>22</b> |
| 5.1 Clock and Reset                               | 22        |
| 5.2 s_axi — Protected Slave                       | 22        |
| 5.3 m_axi — Protected Master                      | 23        |
| 5.4 s_axi_ctrl — Control and Status Slave         | 24        |
| 5.5 irq                                           | 24        |
| <b>6. Register Map</b>                            | <b>25</b> |
| 6.1 CTRL (0x00)                                   | 25        |
| 6.2 STATUS (0x04)                                 | 26        |
| 6.3 IRQ_ENABLE (0x08)                             | 26        |
| 6.4 TIMEOUT_VALUE (0x0C)                          | 26        |
| 6.5 FAULT_ADDR (0x10)                             | 27        |
| 6.6 FAULT_INFO (0x14)                             | 27        |
| 6.7 CORE_INFO (0x18)                              | 27        |
| 6.8 RECOVERY (0x1C)                               | 28        |
| 6.9 Rule Table                                    | 28        |

|                                                 |           |
|-------------------------------------------------|-----------|
| <b>7. Software Programming Model</b>            | <b>29</b> |
| 7.1 Initialisation                              | 29        |
| 7.2 Changing a Rule at Run Time                 | 30        |
| 7.3 Interrupt Service Routine                   | 31        |
| 7.4 Recovery from a Timeout                     | 31        |
| 7.5 Migrating from Version 1.x                  | 33        |
| <b>8. Verification</b>                          | <b>34</b> |
| 8.1 Verification Environment                    | 34        |
| 8.2 Results                                     | 34        |
| 8.3 The Orphan-Response Measurement             | 36        |
| 8.4 Hardware Demonstration                      | 36        |
| 8.5 What Has Not Been Verified                  | 37        |
| <b>9. Design Considerations and Limitations</b> | <b>38</b> |
| 9.1 Single Outstanding Transaction              | 38        |
| 9.2 No Autonomous Flushing                      | 38        |
| 9.3 Address-Only Policy                         | 38        |
| 9.4 No Protocol Legality Checking               | 38        |
| 9.5 Rule Table Is Not Lockable                  | 38        |
| 9.6 Combinational Rule Lookup                   | 39        |
| 9.7 Shared Fault Capture                        | 39        |
| 9.8 Synchronous Reset Only                      | 39        |
| <b>10. Document Revision History</b>            | <b>40</b> |

# 1. About the AXI4-Lite Firewall IP Core

The AXI4-Lite Firewall IP core is a bump-in-the-wire component that sits between a bus master and a peripheral you want to protect. It enforces an allow-list on every transaction and guarantees the master always receives a response, even when the protected peripheral stops answering.

There is no equivalent core in the Intel FPGA IP catalog. AMD/Xilinx ship an AXI Protocol Firewall, but that core polices protocol legality and has no address-range access control; the two are complementary rather than equivalent. See [Section 9](#) for a detailed comparison.

The core is delivered as synthesisable SystemVerilog with a Platform Designer component description, a self-checking testbench, SystemVerilog assertions, and both a Questa and a licence-free Verilator simulation flow.

## 1.1 Features

- **Address-range access control.** Up to 64 software-programmable address ranges, each with independent read and write permission. Default-deny: a transaction matching no rule is rejected.
- **Fault isolation.** A programmable round-trip timeout on every forwarded transaction. On expiry the core answers the master itself, so a wedged peripheral cannot hang the bus master.
- **Separate control port.** Configuration and status live on a physically distinct AXI4-Lite slave, so recovery is never blocked by the fault being recovered from.
- **Interrupt output.** Level interrupt, asserted while any enabled sticky fault bit is set.
- **Isolation control.** Software can isolate the peripheral manually, and the core can isolate it automatically on a timeout.
- **Full status visibility.** Outstanding-transaction and stuck-command status bits let a driver sequence recovery correctly.
- **Secure by default.** Access control is enabled and the rule table is empty out of reset, so an unconfigured core denies everything rather than passing everything.

## 1.2 Device Family Support

The core is written in device-independent synthesisable SystemVerilog. It instantiates no family-specific primitives, no vendor macros, no PLLs and no memory blocks — the rule table is implemented in registers. It should therefore compile for any Intel FPGA family supported by your Quartus Prime installation.

**Table 1. Device Family Support**

| Device family           | Support level                                 |
|-------------------------|-----------------------------------------------|
| All Intel FPGA families | Expected to compile; <b>not characterised</b> |

### Note

No device family has been characterised. The core has not been compiled by Quartus Prime at the time of writing — it has been verified only by simulation and static elaboration. See [Section 8](#) for exactly what has and has not been established.

## 1.3 Resource Utilization

Characterised on Intel MAX 10 by the DE10-Lite example designs in [example/](#), which instantiate the core inside complete, timing-closed systems that have been run on physical hardware.

**Table 2. Resource Utilization**

| Item                              | Logic elements | Dedicated registers |
|-----------------------------------|----------------|---------------------|
| axi_firewall_top, ADDR_WIDTH = 32 | 1,908          | 768                 |
| of which axi_firewall_regs        | 1,657          | 618                 |
| axi_firewall_top, ADDR_WIDTH = 12 | 1,391          | 544                 |
| of which axi_firewall_regs        | 1,050          | 327                 |

Conditions: Intel MAX 10 10M50DAF484C7G (speed grade 7), Quartus Prime 18.1.1 Standard Edition, DATA\_WIDTH = 32, CTRL\_ADDR\_WIDTH = 12, NUM\_RULES = 8, TIMEOUT\_WIDTH = 20. The core infers no memory blocks, multipliers or PLLs.

The two pairs of rows differ only in ADDR\_WIDTH, and the core is 26 % smaller at 12 bits than at 32. That is direct evidence for the area analysis below: the rule table and its comparators dominate, and both scale with the address width.

$f_{MAX}$  on the slow 1200 mV 85 °C model is **60.01 MHz** at ADDR\_WIDTH = 32 (in a system closed at 50 MHz, +3.336 ns of setup slack) and **112.57 MHz** at ADDR\_WIDTH = 12 (closed at 100 MHz, +1.117 ns). The 100 MHz result needs Quartus's High Performance Effort and physical synthesis; at default settings that design reaches 95.27 MHz and misses by 0.496 ns.

The critical path lies inside the core and is the one predicted from the source: captured\_awaddr to wr\_timeout\_cnt, which is the combinational rule lookup feeding the forward-or-reject decision that gates the timeout counter.

The two structures that dominate area and timing are predictable from the source, and both are now confirmed by measurement:

- **The rule table** is NUM\_RULES × (2 × ADDR\_WIDTH + 3) registers. At the default NUM\_RULES = 8 and ADDR\_WIDTH = 32 that is 536 registers, which accounts for most of the 618 measured in axi\_firewall\_regs; the remainder are the control, status, interrupt and fault-capture registers.
- **The rule lookup** is a purely combinational priority chain over NUM\_RULES entries, duplicated for the read and write paths. Each entry is two ADDR\_WIDTH-wide magnitude comparisons. This is the measured critical path and the term that scales with NUM\_RULES.

#### Note

These are measurements at two address widths on one device and speed grade, at a single value of NUM\_RULES.  $f_{MAX}$  as a function of NUM\_RULES, and behaviour on any other device family, remain uncharacterised. If the lookup limits  $f_{MAX}$  in your system, the standard remedy is to register the lookup result and accept one extra cycle of latency in the EVAL state.

## 1.4 Release Information

**Table 3. Release Information**

| Item                    | Description                   |
|-------------------------|-------------------------------|
| Version                 | 2.0                           |
| CORE_INFO version field | 0x0200                        |
| Release date            | August 2026                   |
| Ordering code           | None — source-available       |
| Vendor                  | monkstein88                   |
| IP catalog name         | AXI4-Lite Firewall            |
| IP catalog group        | Bridges and Adapters / Custom |
| Language                | SystemVerilog (IEEE 1800)     |

**Caution**

Version 2.0 is a breaking change from 1.x. The peripheral reset output was removed and the recovery procedure changed. A 1.x driver run against a 2.0 core will acknowledge a fault and then see every subsequent transaction return SLVERR. Read the version field in `CORE_INFO` before assuming either behaviour. See [Section 7.5](#).

## 2. Getting Started

### 2.1 Installing the IP Core

The core is distributed as source. To make it visible to Platform Designer:

1. Clone or copy the repository to your machine.
2. In Quartus Prime, click **Tools ► Options ► IP Catalog Search Locations**.
3. Add the repository's **top-level directory** — the one containing `altera_axi4_lite_firewall/` — to the search path.
4. In Platform Designer, click **File ► Refresh System**.

The core appears in the IP Catalog as **AXI4-Lite Firewall** under *Bridges and Adapters / Custom*.

#### Note

`_hw.tcl` syntax has changed across Quartus Prime releases, and between Standard and Pro editions. The supplied `axi_firewall_hw.tcl` has not been validated against any specific release. If the component fails to import, see [Section 2.1.1](#).

#### 2.1.1 If the Component Does Not Import

Package the component manually. This takes a few minutes and produces a `_hw.tcl` guaranteed correct for your installed toolchain:

1. In Platform Designer, click **File ► New Component**.
2. On the **Files** tab, add `rtl/axi_firewall_regs.sv` and `rtl/axi_firewall_top.sv` as synthesis files. Set `axi_firewall_top.sv` as the top-level file.
3. Click **Analyze Synthesis Files**.
4. On the **Signals & Interfaces** tab, confirm the grouping. Every port follows the `s_axi_*` / `m_axi_*` / `s_axi_ctrl_*` convention with standard AXI4-Lite suffixes, so signal analysis should infer three AXI4-Lite interfaces plus clock, reset and interrupt automatically. Correct anything it misses.
5. Click **Finish** and save.

#### Caution

Declare the RTL as **SystemVerilog**, not Verilog. The source uses `logic`, `always_ff`, packed structs and enumerated types. Analysed with the Verilog-2001 parser it fails on the first `logic` declaration.

### 2.2 Adding the Core to a Platform Designer System

Instantiate the core between the master and the peripheral to be protected:

- Connect the master's data master to `s_axi` and to `s_axi_ctrl`.
- Connect `m_axi` to the protected peripheral.
- Connect `irq` to an interrupt input on the CPU.
- Connect `clock` and `reset` to the system clock and reset.

No explicit bridge is required between Avalon-MM and AXI4-Lite. Platform Designer's interconnect provides the bridging logic. An *AXI Bridge Intel FPGA IP* may be instantiated explicitly if you want to trade concurrency for reduced interconnect logic, but it is optional.

Assign `s_axi_ctrl` a base address in the CPU's address map. The rule addresses programmed into the core refer to addresses as seen on `s_axi`, which is the protected peripheral's address decode.

## 2.3 Required Connections

**Table 4. Connection Requirements**

| Connection                                              | Requirement                  | Consequence if omitted                                    |
|---------------------------------------------------------|------------------------------|-----------------------------------------------------------|
| <code>s_axi</code> to the master                        | Required                     | Core does nothing                                         |
| <code>m_axi</code> to the peripheral                    | Required                     | Core does nothing                                         |
| <code>s_axi_ctrl</code> to a master                     | Required                     | Rule table cannot be programmed; core denies all traffic  |
| <code>irq</code> to a CPU interrupt                     | Recommended                  | Faults must be polled instead                             |
| Software-controllable reset on the protected peripheral | <b>Required for recovery</b> | Cannot recover from a timeout without a full system reset |

### Caution

The core does not drive the protected peripheral's reset. It did in version 1.x. Recovery from a timeout still requires that the peripheral be reset, so the reset must be reachable from software — a Platform Designer reset bridge under software control, or a PIO output, both work. Without it a timeout is unrecoverable short of resetting the whole system. See [Section 3.5](#).

## 2.4 Simulating the IP Core

Three flows are supported. All three run the same testbench.

**Verilator** — no licence required, recommended for continuous integration:

```
simulation/verilator/run_sim.sh
```

The script runs a strict elaboration check, lints the RTL, builds, and runs the regression. Its exit status is 0 only if every check passes.

**Questa** — adds functional coverage and assertion non-vacuity reporting:

```
cd simulation/questa
vsim -c -do run_sim.tcl
```

Writes `coverage.ucdb`, `coverage_report.txt` and `run.log`. The assertion and cover-directive results are inside `coverage_report.txt`, under the bound SVA instance.

**Icarus Verilog** — functional tests only; the assertion bind is skipped:

```
iverilog -g2012 -DICARUS -o tb.out \
    rtl/axi_firewall_regs.sv rtl/axi_firewall_top.sv tb/axi_firewall_tb.sv
vvp tb.out
```

### Note

Use `-g2012`. The sources are SystemVerilog.



## 2.5 Files Provided

**Table 5. Files Provided**

| File                                            | Description                                                                                                                                                                                                                                                           |
|-------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>rtl/axi_firewall_top.sv</code>            | Top level: datapaths, timeout, isolation, block/unblock                                                                                                                                                                                                               |
| <code>rtl/axi_firewall_regs.sv</code>           | Register block: rule table, status, interrupt, control slave                                                                                                                                                                                                          |
| <code>axi_firewall_hw.tcl</code>                | Platform Designer component description                                                                                                                                                                                                                               |
| <code>tb/axi_firewall_tb.sv</code>              | Self-checking testbench, 103 checks                                                                                                                                                                                                                                   |
| <code>tb/axi_firewall_sva.sv</code>             | SystemVerilog assertions and cover points                                                                                                                                                                                                                             |
| <code>verification/orphan_response_tb.sv</code> | Standalone measurement of the recovery hazard                                                                                                                                                                                                                         |
| <code>simulation/questa/run_sim.tcl</code>      | Questa flow with coverage                                                                                                                                                                                                                                             |
| <code>simulation/verilator/run_sim.sh</code>    | Verilator flow                                                                                                                                                                                                                                                        |
| <code>simulation/verilator/slangcheck.py</code> | Strict elaboration gate                                                                                                                                                                                                                                               |
| <code>example/de10_lite_rtl/</code>             | DE10-Lite example design: sixteen self-checking scenarios in synthesisable RTL, a board-level testbench and a Quartus project. See its <code>README.md</code>                                                                                                         |
| <code>example/de10_lite_nios/</code>            | Nios II/f example in a generated Platform Designer system, 33 checks on hardware. See its <code>README.md</code>                                                                                                                                                      |
| <code>HAL/inc/</code> , <code>HAL/src/</code>   | Nios II HAL driver for the core, version 2.0                                                                                                                                                                                                                          |
| <code>inc/</code>                               | Register map alone, usable without the driver                                                                                                                                                                                                                         |
| <code>axi4_lite_firewall_sw.tcl</code>          | BSP driver description. Its presence is what makes the driver automatic: add the component to a Platform Designer system and the BSP compiles the driver, exposes the headers, and constructs each instance in <code>alt_sys_init.c</code> before <code>main()</code> |
| <code>doc/</code>                               | Block diagrams and this user guide                                                                                                                                                                                                                                    |

## 3. Functional Description

---

### 3.1 Block Diagram

The core presents three AXI4-Lite interfaces and one interrupt output.

Traffic flows from the master into `s_axi`, is checked, and is either forwarded to the peripheral through `m_axi` or answered locally with an error response. Configuration and status are reached through `s_axi_ctrl`, which is independent of the data path — so configuring or inspecting the core is never subject to a firewall rule, and never blockable by a peripheral that has stopped answering. Figure 1 shows how the core wires into a system.

Internally the core contains two independent datapaths — one for writes, one for reads — and a shared register block. Each datapath has its own state machine, its own capture registers, its own rule-lookup port and its own fault signals, so a read and a write may be in flight simultaneously. The only shared state is the fault capture pair `FAULT_ADDR` and `FAULT_INFO`. Figure 2 shows the internal structure.

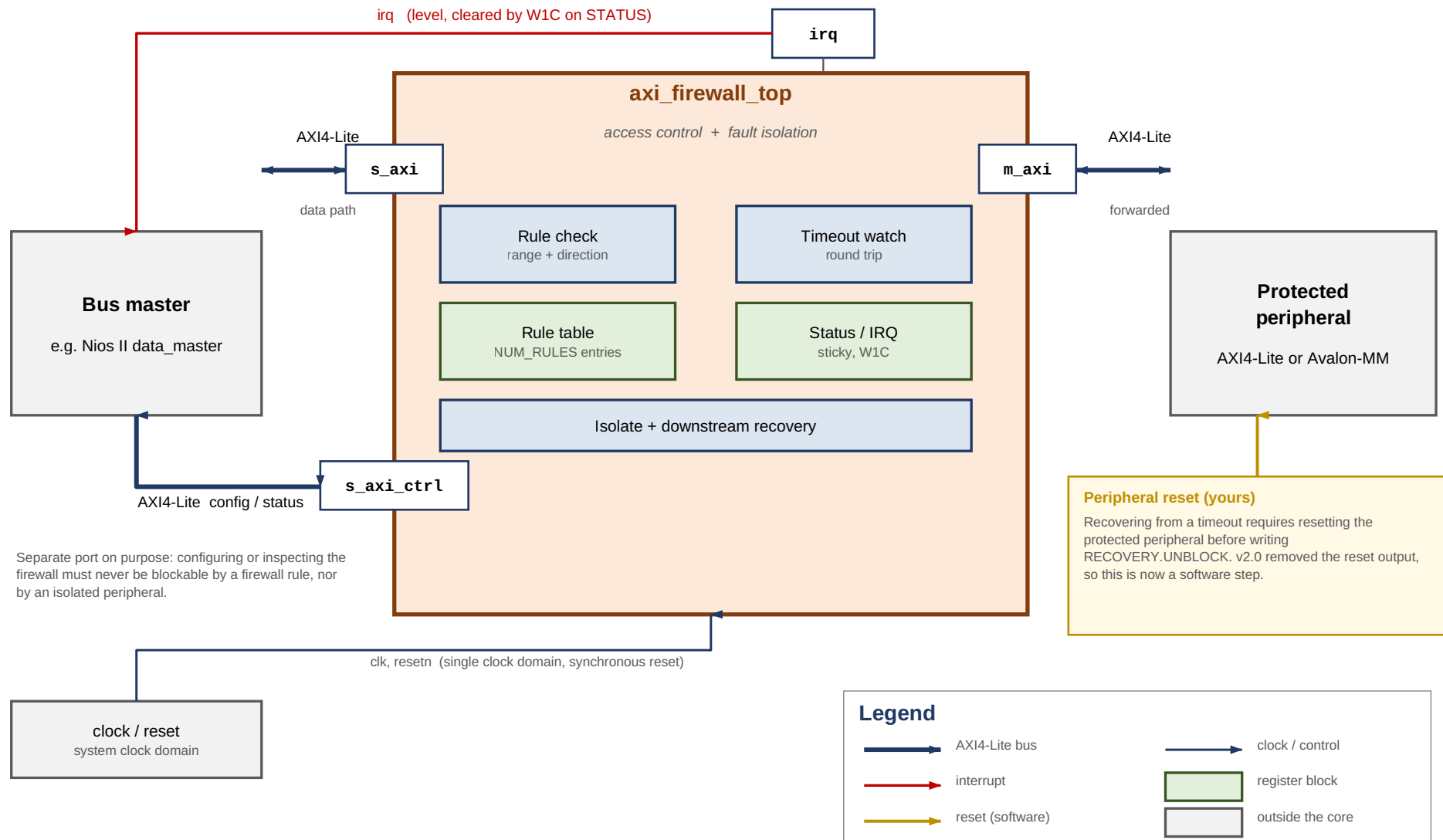


Figure 1. System Context

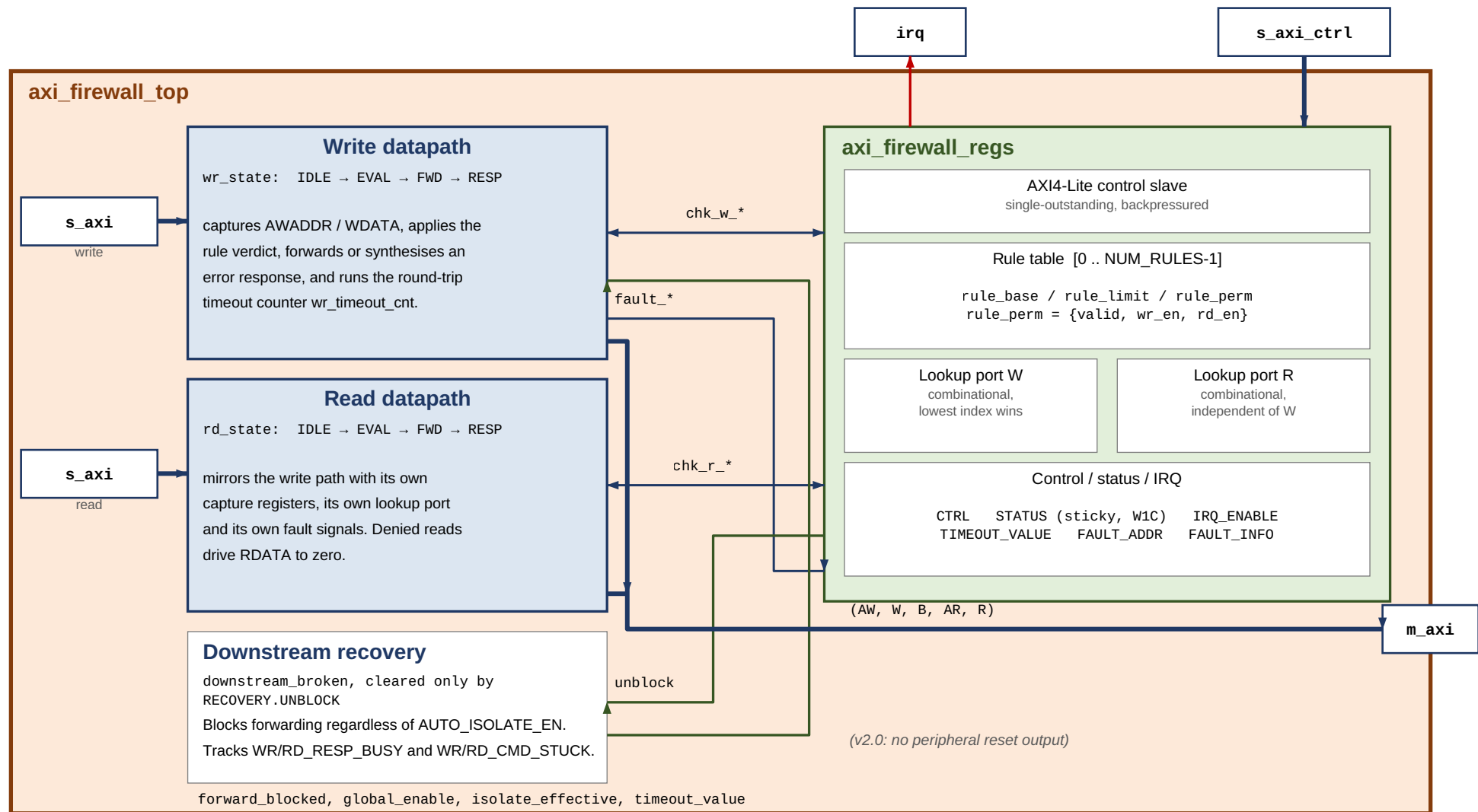


Figure 2. Internal Architecture

## 3.2 Access Control

Every transaction's address is compared against the rule table. Each rule is an inclusive address range with independent read and write permission, and a valid bit.

The verdict is applied in the EVAL state, in strict priority order:

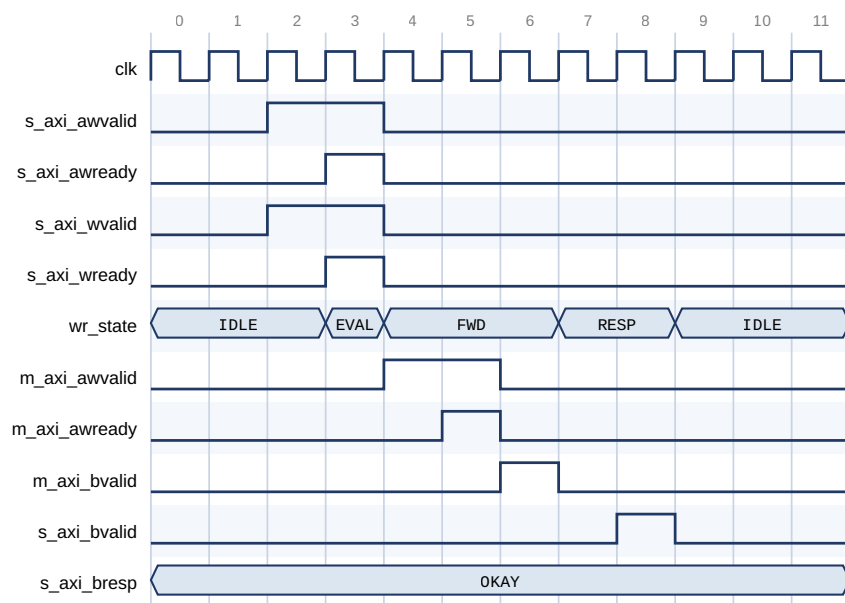
1. If the downstream is blocked or the core is isolated, respond SLVERR.
2. If `CTRL.GLOBAL_ENABLE` is clear, forward unconditionally (bypass mode).
3. If no valid rule contains the address, respond DECERR.
4. If a rule contains the address but does not permit this direction, respond SLVERR.
5. Otherwise, forward the transaction.

**Table 6. Response Encoding**

| Condition                      | Response                      | Status bit set | Interrupt       |
|--------------------------------|-------------------------------|----------------|-----------------|
| Permitted, peripheral answers  | As returned by the peripheral | —              | No              |
| No valid rule matches          | DECERR                        | ADDR_VIOLATION | Yes, if enabled |
| Rule matches, direction denied | SLVERR                        | PERM_VIOLATION | Yes, if enabled |
| Blocked or isolated            | SLVERR                        | None           | No              |
| Peripheral timed out           | SLVERR                        | TIMEOUT_ERROR  | Yes, if enabled |

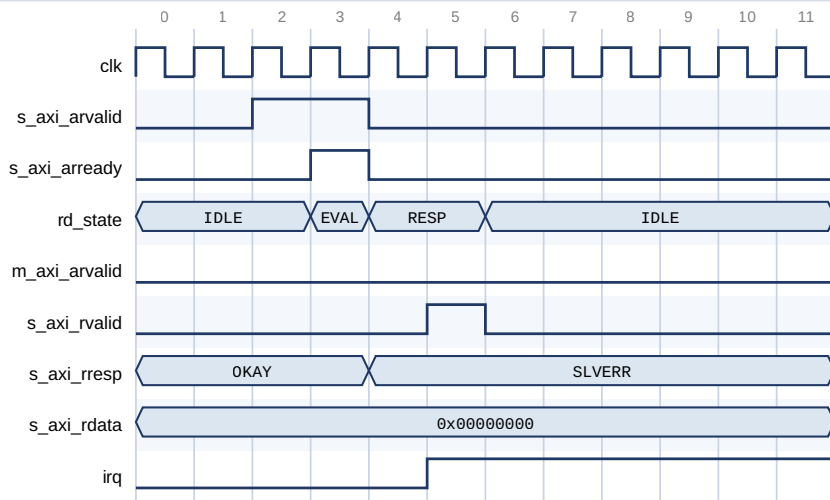
A transaction rejected because the core is isolated or blocked sets no status bit and raises no interrupt. This is deliberate: only genuine policy violations and timeouts are logged, so a burst of rejected traffic during isolation cannot bury the fault that caused it.

A denied read drives `RDATA` to zero rather than leaving the previous value on the bus, so a rejected read cannot leak the result of an earlier permitted one.



The address matches a valid rule that permits writes, so the transaction is forwarded on `m_axi`.  
Cycle 1: request accepted (AWREADY and WREADY together). Cycle 6: BVALID returns OKAY.

**Figure 3. Permitted Write — Request to Response, 6 Cycles**



The address falls in a write-only rule. RD\_EVAL rejects it and answers SLVERR from RD\_RESP.  
 m\_axi\_arvalid never asserts - the peripheral is not touched. RDATA is driven to zero, not left stale.  
 irq asserts because STATUS.PERM\_VIOLATION is sticky and its interrupt is enabled.

**Figure 4. Permission-Denied Read — Answered Locally, Nothing Reaches the Peripheral**

In Figure 4, note that `m_axi_arvalid` never asserts. The peripheral is not touched at all, `RDATA` is driven to zero, and `irq` asserts because the sticky `PERM_VIOLATION` bit was set with its interrupt enabled.

### 3.3 Rule Evaluation

The lookup is a combinational priority chain: the **lowest-index valid rule** containing the address wins. Ranges need not be disjoint — if you use overlapping ranges, place the more specific rule at the lower index.

Two independent lookup ports are provided, one per datapath, so a read and a write are resolved in the same cycle without contending for a shared comparator.

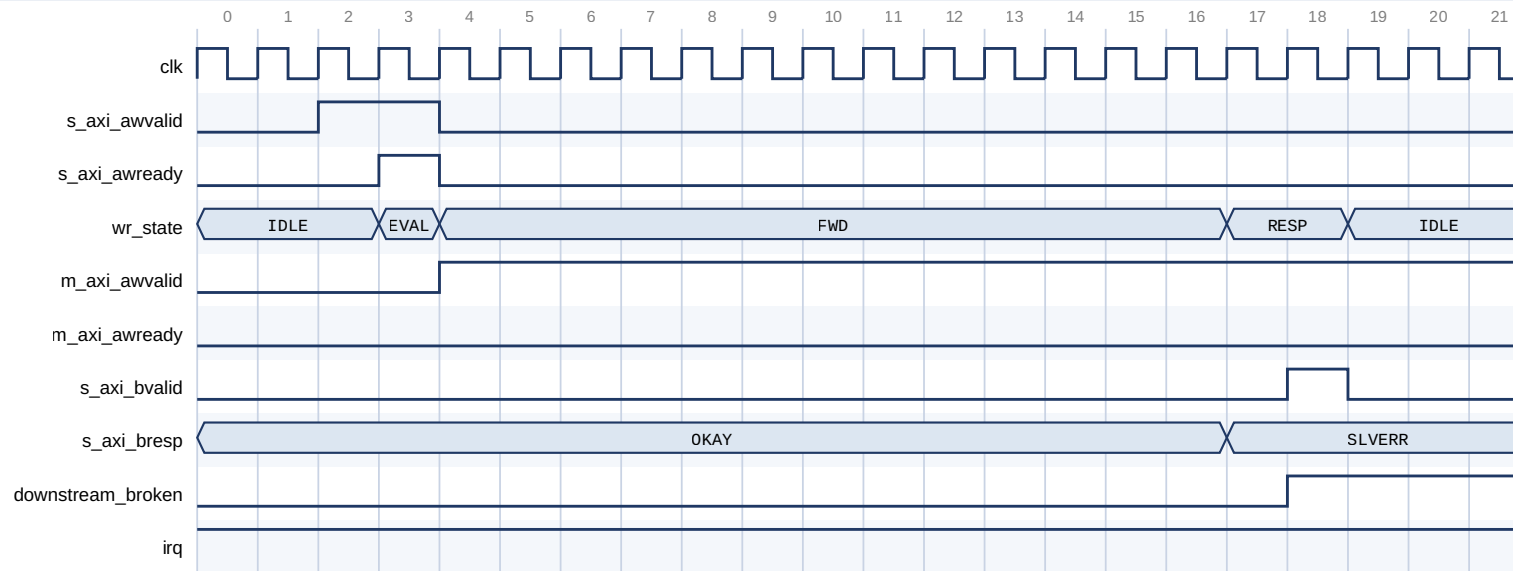
The lookup is purely combinational, so the verdict costs no extra cycle: EVAL is always one cycle.

### 3.4 Fault Isolation and Timeout

Every forwarded transaction is watched by a counter loaded from `TIMEOUT_VALUE`. The counter covers the **entire round trip** — from the moment forwarding begins to the arrival of the response — so it catches both a peripheral that never accepts the command and one that accepts it and then goes quiet.

On expiry the core:

1. answers **SLVERR upstream immediately**, so the master never hangs;
2. latches an internal *downstream-blocked* state that blocks all further forwarding, independently of `CTRL.AUTO_ISOLATE_EN`;
3. **leaves any asserted `m_axi_*VALID` asserted**, because AXI requires `VALID` to remain asserted until `READY`.



The peripheral never raises AWREADY. TIMEOUT\_VALUE is 12 clocks in this capture.

On expiry the core answers SLVERR upstream so the master never hangs, and latches downstream\_broken.

m\_axi\_awvalid stays asserted: AXI forbids withdrawing VALID before READY. Only UNBLOCK may drop it.

**Figure 5. Downstream Timeout — Master Answered Immediately, VALID Left Asserted**

**Caution**

Withdrawing an asserted VALID before its handshake is an AXI protocol violation that can wedge the interconnect between the core and the peripheral, not merely the peripheral. The core therefore never does so on timeout. The stuck VALID is withdrawn at exactly one point — the RECOVERY.UNBLOCK write described in [Section 3.5](#).

CTRL.AUTO\_ISOLATE\_EN governs only whether the visible ISOLATED status bit is set. Blocking after a timeout is required for protocol safety and happens either way.

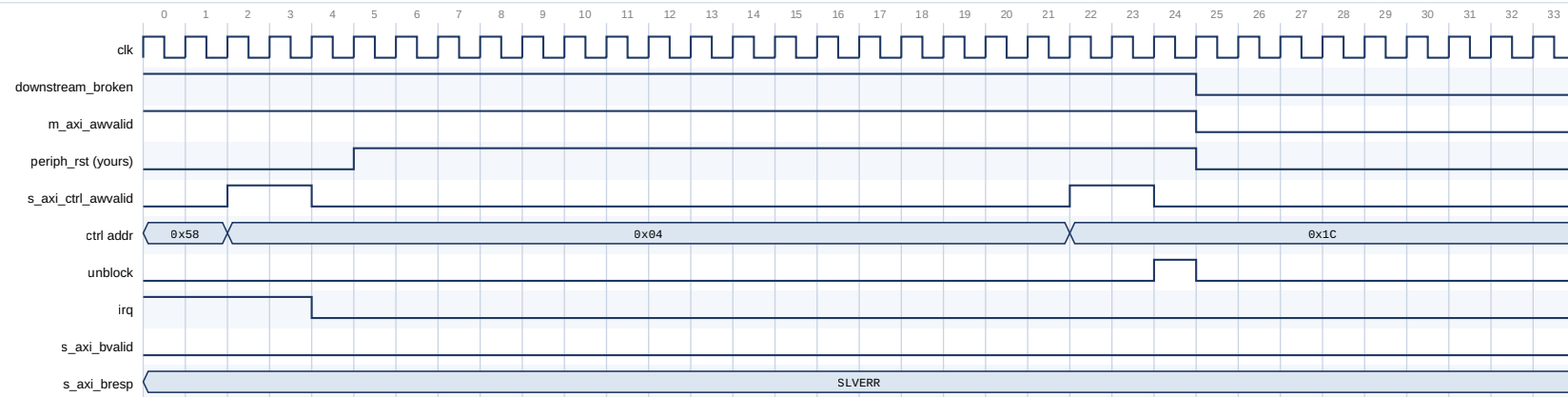
### 3.5 Recovery Sequence

Recovery is an explicit software sequence. It is modelled on the procedure AMD document for their AXI Firewall, which carries the same requirement to reset the monitored side before unblocking.

**Table 7. Recovery Sequence**

| Step | Action                                                                         | Why                                                                                                                                           |
|------|--------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Stop issuing transactions to s_axi                                             | New transactions are rejected while blocked                                                                                                   |
| 2    | Write 1 to the sticky STATUS bits                                              | Acknowledges the fault and releases the auto-isolate latch.<br><b>Required before step 5</b> — see below                                      |
| 3    | Poll STATUS until WR_RESP_BUSY and RD_RESP_BUSY clear — <b>with a bound</b>    | Tells you no late response is still in flight                                                                                                 |
| 4    | <b>Assert the protected peripheral's reset</b> and hold it ( $\geq 16$ clocks) | Discards the peripheral's AXI state                                                                                                           |
| 5    | Write 1 to RECOVERY.UNBLOCK, <b>with the peripheral still held in reset</b>    | Reopens forwarding and withdraws the stuck VALID. Holding reset means nothing downstream can latch the command as it is withdrawn — see below |
| 6    | Release the peripheral's reset                                                 | The peripheral restarts with no knowledge of the abandoned transaction                                                                        |
| 7    | Resume, retrying anything that failed                                          | Transactions attempted while blocked returned SLVERR                                                                                          |





Write 0x04 acknowledges the fault (irq drops). The peripheral reset is driven by the system, not the core.  
 Write 0x1C pulses unblock: downstream\_broken clears and the stuck m\_axi\_awvalid is withdrawn in the same event.  
 The final write completes with OKAY, showing forwarding has reopened.

**Figure 6. Recovery Sequence — Acknowledge, Reset the Peripheral, UNBLOCK**

**Caution**

Step 4 is not optional. `UNBLOCK` causes the core to withdraw an asserted `VALID`. If the peripheral has not been reset, that is a protocol violation on a live bus, and the peripheral may additionally still act on a transaction the core already reported to the master as failed. Measured: 0 of 25 tested timing offsets mis-attribute a stale response when the sequence is followed; 1 of 25 does when step 4 is skipped.

**Caution**

The reset must still be asserted when step 5 executes. Reset is a held state across steps 4 to 6, not a pulse to be fired and forgotten before unblocking.

The reason is that the orphaned command is still on `m_axi` until `UNBLOCK` retracts it. A peripheral that has been reset and *released* sees a perfectly valid `AWVALID`/`ARVALID` waiting on the bus and accepts it, committing a transaction the master was already told had failed. Unlike the stale-response hazard above, which is timing-dependent and was measured at 1 offset in 25, this one is deterministic: it happens every time.

Scenarios `b` and `C` of the DE10-Lite example design are this sequence with the two orderings, and read back `0x00000000` and `0xCAFEF00D` respectively from the protected peripheral.

**Caution**

Step 2 must precede step 5, and the order is load-bearing. Forwarding is gated by two independent conditions — the blocked state and the isolate state — and each is cleared by a different write. `UNBLOCK` clears only the blocked state. If `AUTO_ISOLATE_EN` was set when the timeout occurred, the auto-isolate latch is also set, and only the `W1C` in step 2 releases it. Writing `UNBLOCK` without having acknowledged `STATUS` first leaves the core isolated, and every transaction still returns `SLVERR`. `STATUS` then reads `BLOCKED = 0` with `ISOLATED = 1`, which is the signature of this mistake.

### 3.5.1 Bounding the Busy Poll

`WR_RESP_BUSY` and `RD_RESP_BUSY` mean *the peripheral has accepted a command and still owes a response*. A peripheral that accepted a command and then died owes one forever, so an unbounded poll in step 3 hangs precisely when recovery matters most.

Treat the bits as advisory:

- **Clear** — no late response can still be in flight; the reset is unambiguously safe.
- **Stuck** — reset anyway, and let `UNBLOCK` discard what is owed.

`WR_CMD_STUCK` and `RD_CMD_STUCK` report the other case: a command the peripheral never accepted at all. These can only be cleared by `UNBLOCK`, so a driver that sees one knows immediately that polling `RESP_BUSY` will not be enough.

**Note**

This differs from AMD's core, which autonomously flushes outstanding transactions when blocked so that its busy bits always reach zero. Removing this caveat by adding autonomous flushing is the most valuable planned enhancement to this core.

## 3.6 Interrupts

`irq` is a level interrupt, asserted while any sticky `STATUS` bit is set and its corresponding `IRQ_ENABLE` bit is set:

```
irq = (ADDR_VIOLATION & IRQ_ENABLE[0])
      | (PERM_VIOLATION & IRQ_ENABLE[1])
      | (TIMEOUT_ERROR & IRQ_ENABLE[2])
```

Clear the interrupt at its source by writing 1 to the relevant `STATUS` bit. There is no separate acknowledge register. This is the standard memory-mapped-peripheral idiom and works directly with the Nios II HAL ISR pattern.

`FAULT_ADDR` and `FAULT_INFO` capture the address and type of the most recent fault of any kind.

#### Caution

If the read and write datapaths fault in the *same cycle* with *different* fault types, the capture registers can disagree with each other. `FAULT_ADDR` and `FAULT_INFO.WAS_WRITE` always take the write side. `FAULT_INFO.FAULT_TYPE` is resolved separately, by type precedence across both datapaths — `TIMEOUT` beats `PERM` beats `ADDR`. So a simultaneous write-`ADDR` and read-`PERM` fault reports `WAS_WRITE = 1` with the write's address, but `FAULT_TYPE = PERM`, which belongs to the read.

The sticky `STATUS` bits are unaffected — every fault sets its own bit correctly. Only the single shared capture pair is ambiguous. Software that must attribute every fault individually should correlate with its own transaction log rather than rely on `FAULT_INFO` alone; see [Section 9.7](#).

## 3.7 Control Port Behaviour

`s_axi_ctrl` is single-outstanding and applies backpressure accordingly: `AWREADY` and `WREADY` are withheld while a `BVALID` is unacknowledged, and `ARREADY` while an `RVALID` is. A master that issues one access at a time sees no difference; one that pipelines simply waits.

Writes to reserved or unaligned offsets are ignored and answered `OKAY`; reads of them return zero.

The control port is deliberately independent of the data path. Configuring or inspecting the core is never subject to a firewall rule, and never blockable by an isolated or hung peripheral — which is what makes recovery possible at all.

## 3.8 Reset

`resetn` is active-low and **synchronous**. It resets the entire core: both datapath state machines, the rule table, all status and configuration registers, and the blocked state.

A global reset is therefore also the escape hatch of last resort for a downstream that never recovers — it clears the blocked state without an `UNBLOCK`, at the cost of losing the rule table, which must be reprogrammed.

Out of reset the core is **secure by default**: `GLOBAL_ENABLE` is 1 and the rule table is empty, so every transaction is denied until rules are programmed.

## 3.9 Latency

**Table 8. Latency, Zero-Wait-State Peripheral**

| Operation                                             | Cycles |
|-------------------------------------------------------|--------|
| Single write, request asserted to <code>BVALID</code> | 6      |
| Single read, request asserted to <code>RVALID</code>  | 6      |

Measured in simulation against a zero-wait-state peripheral model, counting clock edges from request assertion to response valid. The testbench fails the run if either exceeds 8 cycles, so the figures cannot silently regress.

The core is single-outstanding and non-pipelined, so this cost is per transaction and does not amortise.

## 4. Parameters

All five parameters are compile-time. Nothing in the core is runtime-reconfigurable except the rule table and the control registers.

**Table 9. Parameters**

| Parameter       | Type    | Default | Legal range | Description                                                                                                                      |
|-----------------|---------|---------|-------------|----------------------------------------------------------------------------------------------------------------------------------|
| ADDR_WIDTH      | Integer | 32      | 8–32        | Width of <code>s_axi / m_axi</code> address buses. Also the width of every rule base and limit, and of <code>FAULT_ADDR</code> . |
| DATA_WIDTH      | Integer | 32      | 32, 64      | Width of <code>s_axi / m_axi</code> data buses. <code>WSTRB</code> is <code>DATA_WIDTH/8</code> bits.                            |
| CTRL_ADDR_WIDTH | Integer | 12      | 8–16        | Width of the <code>s_axi_ctrl</code> address bus.                                                                                |
| NUM_RULES       | Integer | 8       | 1–64        | Number of address-range rules.                                                                                                   |
| TIMEOUT_WIDTH   | Integer | 20      | 8–32        | Width of the timeout counter. Maximum programmable timeout is $2^{\text{TIMEOUT\_WIDTH}} - 1$ clock cycles.                      |

Ranges are enforced by `axi_firewall_hw.tcl`. `DATA_WIDTH` is restricted to 32 and 64 because AXI4-Lite defines only those two.

### Sizing CTRL\_ADDR\_WIDTH

The register file spans `0x40 + NUM_RULES × 16` bytes, so:

$$\text{CTRL\_ADDR\_WIDTH} \geq \text{ceil}(\log_2(0x40 + \text{NUM\_RULES} \times 16))$$

**Table 10. Minimum CTRL\_ADDR\_WIDTH**

| NUM_RULES   | Table spans | Highest byte used | Minimum CTRL_ADDR_WIDTH |
|-------------|-------------|-------------------|-------------------------|
| 1           | 0x50 bytes  | 0x4F              | 7                       |
| 4           | 0x80 bytes  | 0x7F              | 7                       |
| 8 (default) | 0xC0 bytes  | 0xBF              | 8                       |
| 16          | 0x140 bytes | 0x13F             | 9                       |
| 32          | 0x240 bytes | 0x23F             | 10                      |
| 64          | 0x440 bytes | 0x43F             | 11                      |

The default of 12 covers every legal `NUM_RULES`. A validation callback in `axi_firewall_hw.tcl` rejects an undersized value, because the failure mode is silent: high-index rules simply become unreachable, the rule table looks like it programmed correctly, and the firewall quietly enforces fewer rules than you configured.

### Sizing TIMEOUT\_WIDTH

Choose the timeout to be comfortably longer than the protected peripheral's worst-case response, including any interconnect arbitration. `TIMEOUT_WIDTH` only has to be wide enough to express it.

**Table 11. Maximum Timeout by Width**

| TIMEOUT_WIDTH | Maximum count | At 100 MHz  | At 50 MHz   |
|---------------|---------------|-------------|-------------|
| 8             | 255           | 2.6 $\mu$ s | 5.1 $\mu$ s |
| 12            | 4 095         | 41 $\mu$ s  | 82 $\mu$ s  |
| 16            | 65 535        | 655 $\mu$ s | 1.3 ms      |
| 20 (default)  | 1 048 575     | 10.5 ms     | 21 ms       |
| 24            | 16 777 215    | 168 ms      | 336 ms      |
| 32            | 4 294 967 295 | 43 s        | 86 s        |

**Note**

TIMEOUT\_VALUE resets to all ones, which at the default width is 10.5 ms at 100 MHz — long enough that the timeout is effectively disabled until software programs a real value. This is deliberate: the reset state denies traffic on policy grounds, but never manufactures a spurious timeout during boot.

## 5. Interface Signals

The core has 60 ports across six Platform Designer interfaces.

**Table 12. Interfaces**

| Interface  | Type             | Role                                 | Signals |
|------------|------------------|--------------------------------------|---------|
| clock      | Clock sink       | Single clock domain                  | 1       |
| reset      | Reset sink       | Active-low, synchronous              | 1       |
| s_axi      | AXI4-Lite slave  | Protected data path, master side     | 19      |
| m_axi      | AXI4-Lite master | Protected data path, peripheral side | 19      |
| s_axi_ctrl | AXI4-Lite slave  | Configuration and status             | 19      |
| irq        | Interrupt sender | Fault interrupt                      | 1       |

### 5.1 Clock and Reset

**Table 13. Clock and Reset Signals**

| Signal | Direction | Width | Description                                                                         |
|--------|-----------|-------|-------------------------------------------------------------------------------------|
| clk    | Input     | 1     | Clock. All three interfaces are synchronous to it; there is no CDC in the core.     |
| resetn | Input     | 1     | Active-low <b>synchronous</b> reset. Resets the FSMs, rule table and all registers. |

### 5.2 s\_axi — Protected Slave

Connect to the bus master. Standard AXI4-Lite slave.

**Table 14. s\_axi Signals**

| Signal        | Direction | Width        | Description                                                                            |
|---------------|-----------|--------------|----------------------------------------------------------------------------------------|
| s_axi_awaddr  | Input     | ADDR_WIDTH   | Write address                                                                          |
| s_axi_awprot  | Input     | 3            | Protection type. Accepted and forwarded unchanged; <b>not</b> used in rule evaluation. |
| s_axi_awvalid | Input     | 1            | Write address valid                                                                    |
| s_axi_awready | Output    | 1            | Write address ready                                                                    |
| s_axi_wdata   | Input     | DATA_WIDTH   | Write data                                                                             |
| s_axi_wstrb   | Input     | DATA_WIDTH/8 | Write strobes                                                                          |
| s_axi_wvalid  | Input     | 1            | Write data valid                                                                       |
| s_axi_wready  | Output    | 1            | Write data ready                                                                       |
| s_axi_bresp   | Output    | 2            | Write response: OKAY , SLVERR or DECERR                                                |
| s_axi_bvalid  | Output    | 1            | Write response valid                                                                   |
| s_axi_bready  | Input     | 1            | Write response ready                                                                   |
| s_axi_araddr  | Input     | ADDR_WIDTH   | Read address                                                                           |
| s_axi_arprot  | Input     | 3            | Protection type. Forwarded unchanged; not used in rule evaluation.                     |
| s_axi_arvalid | Input     | 1            | Read address valid                                                                     |
| s_axi_arready | Output    | 1            | Read address ready                                                                     |
| s_axi_rdata   | Output    | DATA_WIDTH   | Read data. Driven to zero on any denied read.                                          |
| s_axi_rresp   | Output    | 2            | Read response                                                                          |
| s_axi_rvalid  | Output    | 1            | Read data valid                                                                        |
| s_axi_rready  | Input     | 1            | Read data ready                                                                        |

**Note**

AWPROT / ARPROT are transported but not policed. The rule table keys on address and direction only. If you need privilege-based filtering, that is a feature addition, not a configuration option.

### 5.3 m\_axi — Protected Master

Connect to the peripheral being protected. Signal names and widths mirror s\_axi with directions reversed.

**Table 15. m\_axi Signals of Note**

| Signal        | Direction | Width      | Description                                                 |
|---------------|-----------|------------|-------------------------------------------------------------|
| m_axi_awaddr  | Output    | ADDR_WIDTH | Forwarded write address, unmodified                         |
| m_axi_awvalid | Output    | 1          | Held asserted until AWREADY , including across a timeout    |
| m_axi_wvalid  | Output    | 1          | Held asserted until WREADY , including across a timeout     |
| m_axi_bready  | Output    | 1          | <b>Tied high.</b> The core always accepts a write response. |
| m_axi_rready  | Output    | 1          | <b>Tied high.</b> The core always accepts read data.        |

**Caution**

`m_axi_bready` and `m_axi_rready` are tied high so that a peripheral can never be back-pressured into a deadlock by the firewall itself. The consequence is that a late response from an abandoned transaction is accepted rather than rejected, which is exactly why the recovery sequence resets the peripheral before unblocking. See [Section 3.5](#).

## 5.4 `s_axi_ctrl` — Control and Status Slave

A separate AXI4-Lite slave carrying the register file. `WDATA` is always 32 bits wide and `WSTRB` always 4 bits, independent of `DATA_WIDTH`; the addressable range is set by `CTRL_ADDR_WIDTH`.

**Table 16. `s_axi_ctrl` Signals of Note**

| Signal                         | Direction | Width                        | Description                                                                    |
|--------------------------------|-----------|------------------------------|--------------------------------------------------------------------------------|
| <code>s_axi_ctrl_awaddr</code> | Input     | <code>CTRL_ADDR_WIDTH</code> | Register byte offset                                                           |
| <code>s_axi_ctrl_wdata</code>  | Input     | 32                           | Always 32 bits                                                                 |
| <code>s_axi_ctrl_wstrb</code>  | Input     | 4                            | Byte strobes; honoured for partial-width writes                                |
| <code>s_axi_ctrl_bresp</code>  | Output    | 2                            | Always <code>OKAY</code> ; writes to reserved offsets are ignored, not errored |
| <code>s_axi_ctrl_rdata</code>  | Output    | 32                           | Reserved offsets read as zero                                                  |

## 5.5 `irq`

**Table 17. `irq`**

| Signal           | Direction | Width | Description                                                                                                                                   |
|------------------|-----------|-------|-----------------------------------------------------------------------------------------------------------------------------------------------|
| <code>irq</code> | Output    | 1     | Active-high <b>level</b> interrupt. Asserted while any enabled sticky <code>STATUS</code> bit is set. Cleared by W1C on <code>STATUS</code> . |



## 6. Register Map

All registers are 32 bits and word-aligned on `s_axi_ctrl`. Offsets are byte offsets from the base address assigned to `s_axi_ctrl`.

**Table 18. Register Map**

| Offset        | Name            | Access  | Reset    | Description                                          |
|---------------|-----------------|---------|----------|------------------------------------------------------|
| 0x00          | CTRL            | R/W     | 0x3      | Global enable, auto-isolate, manual isolate          |
| 0x04          | STATUS          | R / W1C | 0x0      | Sticky faults, live isolation and outstanding status |
| 0x08          | IRQ_ENABLE      | R/W     | 0x7      | Per-fault interrupt enables                          |
| 0x0C          | TIMEOUT_VALUE   | R/W     | all ones | Round-trip timeout, in clock cycles                  |
| 0x10          | FAULT_ADDR      | R       | 0x0      | Address of the most recent fault                     |
| 0x14          | FAULT_INFO      | R       | 0x0      | Type and direction of the most recent fault          |
| 0x18          | CORE_INFO       | R       | —        | Version and NUM_RULES                                |
| 0x1C          | RECOVERY        | W       | —        | UNBLOCK . Reads as zero.                             |
| 0x20–0x3F     | <i>reserved</i> | —       | —        | Writes ignored, reads return zero                    |
| 0x40 + i·0x10 | RULE_BASE[i]    | R/W     | 0x0      | Inclusive range base                                 |
| 0x44 + i·0x10 | RULE_LIMIT[i]   | R/W     | 0x0      | Inclusive range limit                                |
| 0x48 + i·0x10 | RULE_PERM[i]    | R/W     | 0x0      | Permissions and valid bit                            |
| 0x4C + i·0x10 | <i>reserved</i> | —       | —        | Padding to a 16-byte stride                          |

i runs 0 to NUM\_RULES – 1 . With the default 8 rules the table spans 0x40–0xBF.

### 6.1 CTRL (0x00)

**Table 19. CTRL**

| Bit  | Name            | Access | Reset | Description                                                       |
|------|-----------------|--------|-------|-------------------------------------------------------------------|
| 31:3 | reserved        | R      | 0     | Reads zero                                                        |
| 2    | MANUAL_ISOLATE  | R/W    | 0     | 1 = reject all traffic with SLVERR , regardless of the rule table |
| 1    | AUTO_ISOLATE_EN | R/W    | 1     | 1 = a timeout also sets the visible ISOLATED status               |
| 0    | GLOBAL_ENABLE   | R/W    | 1     | 1 = enforce the rule table; 0 = forward everything unchecked      |

#### Caution

Clearing GLOBAL\_ENABLE disables access control entirely — the core becomes a transparent pass-through with only the timeout still active. It is a bring-up and debug aid, not an operating mode.

## 6.2 STATUS (0x04)

Table 20. STATUS

| Bit  | Name           | Access | Reset | Description                                                                                                                       |
|------|----------------|--------|-------|-----------------------------------------------------------------------------------------------------------------------------------|
| 31:9 | reserved       | R      | 0     | Reads zero                                                                                                                        |
| 8    | RD_CMD_STUCK   | R      | 0     | Blocked <b>and</b> <code>m_axi_arvalid</code> still asserted — a read the peripheral never accepted                               |
| 7    | WR_CMD_STUCK   | R      | 0     | Blocked <b>and</b> <code>m_axi_awvalid</code> or <code>m_axi_wvalid</code> still asserted — a write the peripheral never accepted |
| 6    | RD_RESP_BUSY   | R      | 0     | The peripheral accepted a read command and still owes <code>RVALID</code>                                                         |
| 5    | WR_RESP_BUSY   | R      | 0     | The peripheral accepted a full write and still owes <code>BVALID</code>                                                           |
| 4    | BLOCKED        | R      | 0     | Forwarding is blocked after a timeout; cleared only by <code>RECOVERY.UNBLOCK</code>                                              |
| 3    | ISOLATED       | R      | 0     | <code>MANUAL_ISOLATE</code> or the auto-isolate latch is active                                                                   |
| 2    | TIMEOUT_ERROR  | R/W1C  | 0     | Sticky: a forwarded transaction timed out                                                                                         |
| 1    | PERM_VIOLATION | R/W1C  | 0     | Sticky: a rule matched but denied the direction                                                                                   |
| 0    | ADDR_VIOLATION | R/W1C  | 0     | Sticky: no valid rule matched the address                                                                                         |

Bits 8:3 are live status, not sticky, and cannot be written. Bits 2:0 are write-1-to-clear; writing 0 to a bit leaves it unchanged.

### Caution

Writing 1 to `TIMEOUT_ERROR` acknowledges the fault and releases the auto-isolate latch, but does **not** clear `BLOCKED` and does not resume forwarding. That requires `RECOVERY.UNBLOCK`. In version 1.x the `W1C` alone resumed traffic; the separation exists so that acknowledging a fault cannot accidentally restart traffic toward a peripheral nobody has reset.

## 6.3 IRQ\_ENABLE (0x08)

Table 21. IRQ\_ENABLE

| Bit  | Name       | Access | Reset | Description                               |
|------|------------|--------|-------|-------------------------------------------|
| 31:3 | reserved   | R      | 0     | Reads zero                                |
| 2    | EN_TIMEOUT | R/W    | 1     | Enable irq on <code>TIMEOUT_ERROR</code>  |
| 1    | EN_PERM    | R/W    | 1     | Enable irq on <code>PERM_VIOLATION</code> |
| 0    | EN_ADDR    | R/W    | 1     | Enable irq on <code>ADDR_VIOLATION</code> |

Masking a bit suppresses the interrupt only. The corresponding `STATUS` bit is still set and still requires a `W1C` to clear.

## 6.4 TIMEOUT\_VALUE (0x0C)

Table 22. TIMEOUT\_VALUE

| Bit               | Name     | Access | Reset    | Description                                    |
|-------------------|----------|--------|----------|------------------------------------------------|
| 31:TIMEOUT_WIDTH  | reserved | R      | 0        | Reads zero; writes to these bits are discarded |
| TIMEOUT_WIDTH-1:0 | TIMEOUT  | R/W    | all ones | Round-trip timeout in clock cycles             |

Byte strobes are honoured, so a byte- or halfword-wide write updates only the addressed bytes of the field. Programming 0 makes every forwarded transaction time out immediately; the core does not reject the value.

## 6.5 FAULT\_ADDR (0x10)

Table 23. FAULT\_ADDR

| Bit            | Name     | Access | Reset | Description                      |
|----------------|----------|--------|-------|----------------------------------|
| 31:ADDR_WIDTH  | reserved | R      | 0     | Reads zero when ADDR_WIDTH < 32  |
| ADDR_WIDTH-1:0 | ADDR     | R      | 0     | Address of the most recent fault |

Overwritten by every new fault, including one whose interrupt is masked. Read it before clearing STATUS if you intend to log it.

## 6.6 FAULT\_INFO (0x14)

Table 24. FAULT\_INFO

| Bit  | Name       | Access | Reset | Description                              |
|------|------------|--------|-------|------------------------------------------|
| 31:4 | reserved   | R      | 0     | Reads zero                               |
| 3:1  | FAULT_TYPE | R      | 0     | See table below                          |
| 0    | WAS_WRITE  | R      | 0     | 1 = the faulting transaction was a write |

Table 25. FAULT\_TYPE Encoding

| Value | Name    | Meaning                                                   |
|-------|---------|-----------------------------------------------------------|
| 0     | NONE    | No fault recorded since reset                             |
| 1     | ADDR    | No valid rule matched — answered DECERR                   |
| 2     | PERM    | A rule matched but denied the direction — answered SLVERR |
| 3     | TIMEOUT | The peripheral did not complete in time — answered SLVERR |

Values 4–7 are not generated.

## 6.7 CORE\_INFO (0x18)

Table 26. CORE\_INFO

| Bit   | Name      | Access | Description                             |
|-------|-----------|--------|-----------------------------------------|
| 31:16 | VERSION   | R      | 0x0200 for version 2.0                  |
| 15:8  | reserved  | R      | Reads zero                              |
| 7:0   | NUM_RULES | R      | Rule count this instance was built with |

Read NUM\_RULES at run time rather than hard-coding it, so one driver binary works across instances. Check VERSION before running the recovery sequence: 1.x and 2.0 recover differently.

## 6.8 RECOVERY (0x1C)

Table 27. RECOVERY

| Bit  | Name     | Access             | Description                                                                       |
|------|----------|--------------------|-----------------------------------------------------------------------------------|
| 31:1 | reserved | W                  | Ignored                                                                           |
| 0    | UNBLOCK  | W1S, self-clearing | Writing 1 clears BLOCKED , reopens forwarding and withdraws any stuck m_axi VALID |

Reads return zero. Writing 0 does nothing. New in version 2.0.

### Caution

Only write UNBLOCK after resetting the protected peripheral. This is the one point at which the core withdraws an asserted VALID , which is safe against a peripheral that has just been reset and a protocol violation against one that has not.

## 6.9 Rule Table

Each rule occupies a 16-byte slot at  $0x40 + i \times 0x10$  .

Table 28. Rule Slot Layout

| Sub-offset | Name          | Access | Reset | Description             |
|------------|---------------|--------|-------|-------------------------|
| +0x0       | RULE_BASE[i]  | R/W    | 0     | Inclusive base address  |
| +0x4       | RULE_LIMIT[i] | R/W    | 0     | Inclusive limit address |
| +0x8       | RULE_PERM[i]  | R/W    | 0     | Permissions, see below  |
| +0xC       | reserved      | —      | —     | Padding                 |

Table 29. RULE\_PERM[i]

| Bit  | Name        | Access | Reset | Description                          |
|------|-------------|--------|-------|--------------------------------------|
| 31:3 | reserved    | R      | 0     | Reads zero                           |
| 2    | VALID       | R/W    | 0     | 1 = this rule takes part in matching |
| 1    | WRITE_ALLOW | R/W    | 0     | 1 = permit writes in this range      |
| 0    | READ_ALLOW  | R/W    | 0     | 1 = permit reads in this range       |

A rule matches when VALID is set and  $RULE\_BASE[i] \leq \text{address} \leq RULE\_LIMIT[i]$  . Both bounds are inclusive, so a single-word window is  $BASE == LIMIT$  .  $BASE > LIMIT$  matches nothing; the core does not reject it.

The lowest-index matching valid rule wins. Only its permission bits are consulted — a later, more permissive rule covering the same address has no effect.

RULE\_BASE and RULE\_LIMIT honour byte strobes, so a byte-wide write updates only the addressed bytes. RULE\_PERM is updated only when WSTRB[0] is set.

## 7. Software Programming Model

Register offsets in the examples below are relative to the base address assigned to `s_axi_ctrl`.

```
#define FW_CTRL            0x00
#define FW_STATUS          0x04
#define FW_IRQ_ENABLE      0x08
#define FW_TIMEOUT         0x0C
#define FW_FAULT_ADDR      0x10
#define FW_FAULT_INFO      0x14
#define FW_CORE_INFO       0x18
#define FW_RECOVERY        0x1C
#define FW_RULE(i)         (0x40 + (i) * 0x10)
#define FW_RULE_BASE(i)    (FW_RULE(i) + 0x0)
#define FW_RULE_LIMIT(i)   (FW_RULE(i) + 0x4)
#define FW_RULE_PERM(i)    (FW_RULE(i) + 0x8)

#define FW_PERM_VALID      (1u << 2)
#define FW_PERM_WRITE      (1u << 1)
#define FW_PERM_READ       (1u << 0)

#define FW_ST_ADDR_VIOL    (1u << 0)
#define FW_ST_PERM_VIOL    (1u << 1)
#define FW_ST_TIMEOUT      (1u << 2)
#define FW_ST_ISOLATED     (1u << 3)
#define FW_ST_BLOCKED      (1u << 4)
#define FW_ST_WR_BUSY      (1u << 5)
#define FW_ST_RD_BUSY      (1u << 6)
#define FW_ST_WR_STUCK     (1u << 7)
#define FW_ST_RD_STUCK     (1u << 8)
#define FW_ST_FAULTS       (FW_ST_ADDR_VIOL | FW_ST_PERM_VIOL | FW_ST_TIMEOUT)
```

### 7.1 Initialisation

Program the rule table before allowing any traffic to reach `s_axi`. Out of reset the table is empty and `GLOBAL_ENABLE` is set, so every transaction is denied until you configure it.

```

void fw_init(void *base, uint32_t timeout_cycles)
{
    /* Rules are only meaningful up to the count this instance was built
       with. Read it rather than assuming the default. */
    uint32_t n_rules = IORD_32DIRECT(base, FW_CORE_INFO) & 0xFF;

    for (uint32_t i = 0; i < n_rules; i++)
        IOWR_32DIRECT(base, FW_RULE_PERM(i), 0); /* invalidate */

    IOWR_32DIRECT(base, FW_TIMEOUT, timeout_cycles);
    IOWR_32DIRECT(base, FW_STATUS, FW_ST_FAULTS); /* clear stale sticky */
    IOWR_32DIRECT(base, FW_IRQ_ENABLE, 0x7);
}

void fw_add_rule(void *base, uint32_t i,
                 uint32_t lo, uint32_t hi, int can_read, int can_write)
{
    /* Program the range before the valid bit. Setting VALID first would
       expose a window in which the rule is live over a stale range. */
    IOWR_32DIRECT(base, FW_RULE_BASE(i), lo);
    IOWR_32DIRECT(base, FW_RULE_LIMIT(i), hi);
    IOWR_32DIRECT(base, FW_RULE_PERM(i),
                  FW_PERM_VALID
                    | (can_write ? FW_PERM_WRITE : 0)
                    | (can_read ? FW_PERM_READ : 0));
}

```

### Caution

Write `RULE_PERM` last. The rule table is live: a rule becomes active the instant its `VALID` bit is set, and a transaction arriving in the window between setting `VALID` and updating the range would be judged against whatever the range registers happened to hold.

## 7.2 Changing a Rule at Run Time

To narrow a rule safely, clear `VALID` first, change the range, then set `VALID` again. Widening a rule can be done range-first without an intermediate invalidation, because the intermediate state is never more permissive than either endpoint.

## 7.3 Interrupt Service Routine

```
void fw_isr(void *base)
{
    uint32_t st = IORD_32DIRECT(base, FW_STATUS);

    /* Capture before acknowledging: a later fault overwrites these. */
    uint32_t addr = IORD_32DIRECT(base, FW_FAULT_ADDR);
    uint32_t info = IORD_32DIRECT(base, FW_FAULT_INFO);

    if (st & FW_ST_ADDR_VIOL) log_addr_violation(addr, info & 1);
    if (st & FW_ST_PERM_VIOL) log_perm_violation(addr, info & 1);
    if (st & FW_ST_TIMEOUT)    schedule_recovery();

    /* W1C only the bits actually seen, so a fault raised between the read
       and the write is not silently discarded. */
    IOWR_32DIRECT(base, FW_STATUS, st & FW_ST_FAULTS);
}
```

Do not run the recovery sequence from inside the ISR. It resets the peripheral and polls, which does not belong in interrupt context.

## 7.4 Recovery from a Timeout

This implements the sequence in [Section 3.5](#).

Note the shape of the reset interface: **two functions, not one**. A single `reset_peripheral()` that pulses and returns cannot express this sequence, because the reset has to still be asserted when `UNBLOCK` is written.

```

int fw_recover(void *base,
               void (*reset_assert)(void), /* drive the peripheral's reset */
               void (*reset_release)(void), /* and release it */
               void (*delay_clocks)(unsigned))
{
    uint32_t st;

    /* Step 1 is the caller's: stop issuing transactions to the
       protected peripheral before calling this function. */

    /* Step 2: acknowledge the fault and release the auto-isolate latch.
       This does NOT reopen the downstream - that is step 5. */
    IOWR_32DIRECT(base, FW_STATUS, FW_ST_FAULTS);

    /* Step 3: bounded poll. A peripheral that accepted a command and then
       died owes a response forever, so this MUST NOT be unbounded. On
       expiry, continue anyway - UNBLOCK discards what is still owed. */
    for (int spin = 0; spin < 1000; spin++) {
        st = IORD_32DIRECT(base, FW_STATUS);
        if (!(st & (FW_ST_WR_BUSY | FW_ST_RD_BUSY)))
            break;
    }

    /* Step 4: assert the protected peripheral's reset and HOLD it. Not
       optional. At least 16 clocks so its AXI state machines fully
       re-initialise. */
    reset_assert();
    delay_clocks(16);

    /* Step 5: reopen the downstream, with the peripheral STILL IN RESET.
       UNBLOCK is the point at which the core withdraws the VALID left over
       from the timed-out command. Until then that command is still sitting
       on m_axi, and a peripheral that is out of reset will accept it - see
       the second Caution in Section 3.5. */
    IOWR_32DIRECT(base, FW_RECOVERY, 1);

    /* Step 6: only now release the peripheral. */
    reset_release();

    /* Step 7: the caller retries anything that failed while blocked. */
    return (IORD_32DIRECT(base, FW_STATUS) & FW_ST_BLOCKED) ? -1 : 0;
}

```

#### Note

A tested implementation of this function, with the reset hooks parameterised and the ordering checked by host tests, is provided as `HAL/src/altera_axi4_lite_firewall.c`.



**Caution**

The bound on step 3 is the important part. `WR_RESP_BUSY` and `RD_RESP_BUSY` clear when the peripheral delivers what it owes, and a dead peripheral never will. An unbounded poll hangs the recovery path in exactly the case it exists for. Check `WR_CMD_STUCK` and `RD_CMD_STUCK` too: those can only be cleared by `UNBLOCK`, so seeing either tells you immediately that polling `RESP_BUSY` will not be enough.

## 7.5 Migrating from Version 1.x

**Table 30. Version 1.x to 2.0 Changes**

| Area                                     | Version 1.x                                         | Version 2.0                                                                        |
|------------------------------------------|-----------------------------------------------------|------------------------------------------------------------------------------------|
| Peripheral reset                         | <code>m_axi_resetn</code> output driven by the core | Port removed; resetting the peripheral is a software step                          |
| <code>RESET_HOLD_CYCLES</code> parameter | Present                                             | Removed                                                                            |
| Resuming after a timeout                 | W1C on <code>STATUS.TIMEOUT_ERROR</code>            | W1C then <code>RECOVERY.UNBLOCK</code>                                             |
| <code>RECOVERY</code> register           | Absent                                              | New at 0x1C                                                                        |
| <code>STATUS</code> bits 8:4             | Absent                                              | <code>BLOCKED</code> , <code>WR/RD_RESP_BUSY</code> , <code>WR/RD_CMD_STUCK</code> |
| <code>CORE_INFO</code> version           | 0x0102                                              | 0x0200                                                                             |

Migration steps:

1. Remove the `m_axi_resetn` connection from the Platform Designer system and provide a software-controllable reset to the peripheral instead.
2. Replace every "clear `TIMEOUT_ERROR` to resume" site with the six-step sequence in [Section 7.4](#).
3. Gate the new path on `CORE_INFO[31:16]` if one driver must support both.

A 1.x driver run unchanged against a 2.0 core does not fail loudly. It acknowledges the fault, never writes `UNBLOCK`, and every subsequent transaction returns `SLVERR` from a core that stays blocked forever.

## 8. Verification

The core is verified by simulation and static elaboration. This section states precisely what has been established and what has not.

### 8.1 Verification Environment

Three front ends are used, because each catches things the others miss.

**Table 31. Verification Tools**

| Tool      | Version | Role                                            | What it uniquely caught                    |
|-----------|---------|-------------------------------------------------|--------------------------------------------|
| slang     | 11      | Strict IEEE 1800 elaboration gate               | Mixed <code>timescale</code> between files |
| Verilator | 5.48    | Regression and lint, 9 parameter configurations | Lint findings across the parameter sweep   |
| Questa    | 2024.1  | Functional and assertion coverage               | Declaration-order errors Verilator accepts |

The Questa finding is worth stating plainly: three constructs in the version 2.0 RTL elaborated cleanly under Verilator and were rejected by Questa. A port connection to an identifier declared later in the file creates an implicit net that then collides with the explicit declaration. Verilator tolerated it; Questa was right to refuse. That is why the toolchain has three front ends rather than one.

### 8.2 Results

All figures below are from the version 2.0 sources.

**Table 32. Verification Results**

| Metric                                          | Result                 |
|-------------------------------------------------|------------------------|
| Self-checking testbench checks                  | 103 / 103 pass         |
| SystemVerilog assertions                        | 14 / 14, zero failures |
| Assertions with a non-vacuous pass              | 14 / 14                |
| Cover directives                                | 6 / 6 hit              |
| FSM states covered                              | 8 / 8                  |
| FSM transitions covered                         | 14 / 14                |
| <code>m_axi</code> protocol violations observed | 0                      |
| Total functional coverage (Questa)              | 85.96 %                |
| slang errors                                    | 0                      |
| Verilator lint findings, 9 configurations       | 0                      |

**Table 33. Assertions**

| Assertion                                  | Passes | Checks                                                                                         |
|--------------------------------------------|--------|------------------------------------------------------------------------------------------------|
| <code>a_suppress_illegal_write</code>      | 4      | A denied write never reaches <code>m_axi</code>                                                |
| <code>a_suppress_illegal_read</code>       | 4      | A denied read never reaches <code>m_axi</code>                                                 |
| <code>a_err_on_blocked_write</code>        | 4      | A blocked write is answered with an error, not silence                                         |
| <code>a_err_on_blocked_read</code>         | 4      | Ditto for reads                                                                                |
| <code>a_awvalid_stability</code>           | 23     | Slave-side <code>AWVALID</code> /payload stable until <code>AWREADY</code>                     |
| <code>a_arvalid_stability</code>           | 23     | Slave-side <code>ARVALID</code> stability                                                      |
| <code>a_bvalid_stability</code>            | 6      | <code>BVALID</code> / <code>BRESP</code> stable until <code>BREADY</code>                      |
| <code>a_rvalid_stability</code>            | 6      | <code>RVALID</code> / <code>RDATA</code> / <code>RRESP</code> stable until <code>RREADY</code> |
| <code>a_m_awvalid_stability</code>         | 162    | Master-side <code>AWVALID</code> stability, excluding the <code>unblock</code> discard         |
| <code>a_m_wvalid_stability</code>          | 162    | Master-side <code>WVALID</code> stability                                                      |
| <code>a_m_arvalid_stability</code>         | 119    | Master-side <code>ARVALID</code> stability                                                     |
| <code>a_no_issue_while_blocked</code>      | 382    | No write is issued to <code>m_axi</code> while blocked                                         |
| <code>a_no_read_issue_while_blocked</code> | 429    | No read is issued while blocked                                                                |
| <code>a_block_holds_until_unblock</code>   | 560    | <code>BLOCKED</code> persists until <code>UNBLOCK</code> , never self-clears                   |

Every assertion has a non-zero non-vacuous pass count. This matters: an assertion that only ever passes vacuously is a comment with syntax highlighting. Two assertions in an earlier revision were exactly that, and the fix was adding a test that made the antecedent true, not editing the assertion.

**Table 34. Cover Directives**

| Directive                             | Hits | What it proves the suite reaches                                          |
|---------------------------------------|------|---------------------------------------------------------------------------|
| <code>c_write_denied</code>           | 4    | A permission-denied write actually occurs                                 |
| <code>c_read_denied</code>            | 4    | A permission-denied read actually occurs                                  |
| <code>c_write_decerr</code>           | 3    | An out-of-range write reaching <code>DECERR</code>                        |
| <code>c_read_decerr</code>            | 2    | An out-of-range read reaching <code>DECERR</code>                         |
| <code>c_block_and_recover</code>      | 5    | A full block-and-recover cycle                                            |
| <code>c_unblock_with_stuck_cmd</code> | 3    | <code>UNBLOCK</code> while a command is still stuck on <code>m_axi</code> |

**Table 35. FSM Transition Coverage**

| Transition  | Write FSM | Read FSM | Exercised by                                  |
|-------------|-----------|----------|-----------------------------------------------|
| IDLE → EVAL | 23        | 23       | Every transaction reaching the core           |
| EVAL → FWD  | 17        | 16       | Every permitted transaction                   |
| EVAL → RESP | 5         | 6        | Denied, out-of-range and blocked transactions |
| EVAL → IDLE | 1         | 1        | Reset asserted during EVAL                    |
| FWD → RESP  | 14        | 13       | Normal completion and timeout                 |
| FWD → IDLE  | 3         | 3        | Reset asserted during FWD                     |
| RESP → IDLE | 19        | 19       | Every completed response                      |

## 8.3 The Orphan-Response Measurement

`verification/orphan_response_tb.sv` measures the hazard that motivates step 4 of the recovery sequence. It runs a clean, permitted write, injects one late `BVALID` from an abandoned transaction at offset  $k$  cycles, and sweeps  $k$  over 25 values. If the master ever sees `SLVERR` on a write the peripheral answered `OKAY`, a stale response was mis-attributed.

**Table 36. Orphan-Response Results**

| Recovery sequence                                         | Offsets that mis-attribute |
|-----------------------------------------------------------|----------------------------|
| Peripheral reset before <code>UNBLOCK</code> (documented) | 0 of 25                    |
| <code>UNBLOCK</code> without resetting the peripheral     | 1 of 25                    |

This is why step 4 is stated as mandatory rather than recommended.

## 8.4 Hardware Demonstration

The example design in `example/de10_lite_rtl/` is a second, independent verification vehicle, and covers ground the testbench structurally cannot: it is the only place the core is driven by **synthesisable hardware** rather than by testbench tasks, through the same RTL that is programmed into an FPGA.

A hardware sequencer programs the rule table over `s_axi_ctrl`, issues transactions at `s_axi`, injects faults into the protected peripheral, and checks every response against the register map in Section 6. Sixteen scenarios cover permitted and denied access in both directions, both violation types, both timeout shapes, isolation, the recovery sequence, bypass and interrupt masking.

There are two example designs, and they answer different questions. The RTL one asks whether the core behaves; the Nios II one asks whether it *integrates*, by putting it behind generated Platform Designer interconnect, an Avalon-to-AXI bridge and a processor data cache.

**Table 37. Hardware Demonstration Results**

| Metric                                 | RTL example                   | Nios II example       |
|----------------------------------------|-------------------------------|-----------------------|
| Driven by                              | a hardware sequencer          | C on a Nios II/f      |
| Clock                                  | 50 MHz                        | 100 MHz (PLL)         |
| <code>ADDR_WIDTH</code>                | 32                            | 12                    |
| <b>Passing on a physical DE10-Lite</b> | <b>16 / 16 scenarios</b>      | <b>33 / 33 checks</b> |
| Board-level checks in simulation       | 80 / 80, Questa and Verilator | not applicable        |
| Quartus 18.1.1 compilation             | 0 errors                      | 0 errors              |
| Timing closure                         | met, +3.336 ns                | met, +1.117 ns        |
| Driver host tests                      | not applicable                | 30 / 30               |

Two results from that work are folded back into this document. The resource and  $f_{\text{MAX}}$  figures in Section 1.3 come from it. And the second Caution in Section 3.5 — that the peripheral's reset must still be asserted when `UNBLOCK` is written — was found by building it: scenarios **b** and **C** are the same recovery with the two orderings, and the wrong one deterministically commits a write the master was told had failed.

Both were run on a Terasic DE10-Lite. The RTL example carries an In-System Sources and Probes instance so a script can drive it and read its pass bitmap back over JTAG; the Nios II example reports over its JTAG UART.

**Note**

Running on hardware exposed two defects that simulation had not, and both were in the Nios II *test program* rather than in the core. An interrupt handler that acknowledged faults was clearing the sticky `STATUS` bits before the checking code could read them, and posted writes meant `STATUS` could be sampled before the offending transaction had been evaluated. Neither is reachable in a testbench, which has no interrupt latency, no write posting and no cache. They are recorded here because they are the failure modes a real driver will meet.

## 8.5 What Has Not Been Verified

Stated plainly, because the absence of a result is easy to mistake for a passing one.

**Table 38. Not Verified**

| Item                                                         | Status                                                                                                                                                                                 |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Quartus Prime analysis of the <code>_hw.tcl</code> component | <b>Verified for Quartus 18.1.1 Standard.</b> Platform Designer recognises all six interfaces and all five parameters, and the component generates and synthesises inside a real system |
| Synthesis results (logic elements, registers)                | <b>Measured</b> at <code>NUM_RULES</code> = 8 on MAX 10 — see <a href="#">Section 1.3</a>                                                                                              |
| Timing closure at 50 MHz on MAX 10                           | <b>Met</b> — see <a href="#">Section 1.3</a>                                                                                                                                           |
| $f_{\text{MAX}}$ vs <code>NUM_RULES</code>                   | Never swept. One data point only                                                                                                                                                       |
| Behaviour inside a generated Platform Designer interconnect  | <b>Verified</b> by the Nios II example, on hardware                                                                                                                                    |
| Device families other than MAX 10                            | None characterised                                                                                                                                                                     |
| Behaviour on physical hardware                               | <b>Verified on a Terasic DE10-Lite</b> ( 10M50DAF484C7G ), both examples                                                                                                               |
| Formal property proof                                        | Not attempted; assertions are simulation-only                                                                                                                                          |

Everything in Sections 1 through 7 that describes core behaviour is backed by simulation, and the parts a demonstration can reach are additionally confirmed on silicon. What remains untested is breadth rather than depth: one device family, one speed grade, two address widths, one value of `NUM_RULES`, and no formal proof.

## 9. Design Considerations and Limitations

### 9.1 Single Outstanding Transaction

Each datapath handles one transaction at a time. A read and a write may overlap, but a second write waits for the first to complete. Interconnect generated by Platform Designer will pipeline requests into the firewall, which simply back-pressures.

This is a deliberate simplification. The core answers the master itself on a timeout, and doing that correctly with multiple outstanding transactions requires tracking which responses are still owed and which have been synthesised — the exact bookkeeping that `RESP_BUSY` and `CMD_STUCK` expose in rudimentary form today.

### 9.2 No Autonomous Flushing

When blocked, the core stops forwarding but does not itself complete or discard transactions the peripheral already accepted. The consequence is the bounded poll in [Section 3.5.1](#).

AMD's AXI Protocol Firewall does flush autonomously, which is why its busy bits always reach zero and its recovery has no equivalent caveat. Adding this is the most valuable planned enhancement.

### 9.3 Address-Only Policy

Rules key on address range and direction. `AWPROT / ARPROT` are forwarded but not evaluated, so the core cannot express "privileged accesses only" or "instruction fetches only".

### 9.4 No Protocol Legality Checking

The core does not police AXI legality on `s_axi`. A master that violates VALID stability will not be caught here. This is the main functional difference from AMD's core, which is a protocol checker with no address-based access control. The two are complementary.

**Table 39. Comparison with the AMD AXI Protocol Firewall**

| Capability                                           | This core           | AMD AXI Protocol Firewall |
|------------------------------------------------------|---------------------|---------------------------|
| Address-range access control                         | Yes, up to 64 rules | No                        |
| Read/write permission per range                      | Yes                 | No                        |
| AXI protocol legality checking                       | No                  | Yes, extensive            |
| Timeout on the protected side                        | Yes                 | Yes                       |
| Guaranteed response to the master                    | Yes                 | Yes                       |
| Blocks after a fault until unblocked                 | Yes                 | Yes                       |
| Autonomous flushing when blocked                     | No                  | Yes                       |
| Requires resetting the monitored side before unblock | Yes                 | Yes                       |
| Full AXI4 (bursts, IDs)                              | No, AXI4-Lite only  | Yes                       |

### 9.5 Rule Table Is Not Lockable

Any master that can reach `s_axi_ctrl` can rewrite the rule table, including disabling the firewall through `GLOBAL_ENABLE`. The core protects a peripheral from a misbehaving master on `s_axi`; it does not protect itself from a malicious master on `s_axi_ctrl`. If your threat model includes the latter, place the control port behind something that arbitrates access to it — a second firewall instance is one option.

A one-way lock bit is a natural addition and is not currently implemented.

## 9.6 Combinational Rule Lookup

The lookup is a combinational priority chain over `NUM_RULES` entries, so it is the expected critical path and it scales with `NUM_RULES`. With no synthesis data there is no guidance on where it stops meeting timing. If it limits  $f_{MAX}$ , register the lookup result and accept one more cycle in EVAL.

## 9.7 Shared Fault Capture

`FAULT_ADDR` and `FAULT_INFO` are single registers shared by both datapaths. Simultaneous read and write faults both set their sticky `STATUS` bits correctly, but only one address and one type are captured — and the two are resolved by *different* rules:

**Table 40. Fault Capture Resolution**

| Field                              | Resolved by                                                                                         |
|------------------------------------|-----------------------------------------------------------------------------------------------------|
| <code>FAULT_ADDR</code>            | Write side wins                                                                                     |
| <code>FAULT_INFO.WAS_WRITE</code>  | Write side wins                                                                                     |
| <code>FAULT_INFO.FAULT_TYPE</code> | Type precedence across both datapaths: <code>TIMEOUT</code> > <code>PERM</code> > <code>ADDR</code> |

The two rules agree whenever both datapaths fault with the same type, or when only one faults. They disagree when the datapaths fault in the same cycle with different types: the address and direction then describe the write while the type may describe the read.

This is a consequence of the capture being one register pair fed by three independent fault sources, and it is not currently arbitrated. Software that must attribute every fault individually needs to correlate with its own transaction log. Giving each datapath its own capture pair would remove the ambiguity and is a candidate for a future revision.

## 9.8 Synchronous Reset Only

`resetn` is sampled synchronously. If your reset source is asynchronous, put a reset synchroniser ahead of the core. Platform Designer's reset bridge does this for you.

## 10. Document Revision History

Table 41. Document Revision History

| Document version | Core version | Date        | Changes                                                                                                                                                                                                                                                                                                                                                                                                         |
|------------------|--------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1.2              | 2.0          | August 2026 | Recorded hardware verification on a Terasic DE10-Lite: both example designs run on a physical board (Section 8.4), the Platform Designer component flow works, and the core is now measured at a second ADDR_WIDTH (Section 1.3). Several rows moved out of "not verified". No RTL change.                                                                                                                      |
| 1.1              | 2.0          | August 2026 | Added measured resource and $f_{MAX}$ figures (Section 1.3), which were previously uncharacterised. Corrected the recovery sequence: the protected peripheral's reset must remain asserted while RECOVERY.UNBLOCK is written, and the sequence is now seven steps rather than six (Sections 3.5 and 7.4). Added the hardware demonstration (Section 8.4) and updated what has not been verified. No RTL change. |
| 1.0              | 2.0          | August 2026 | Initial release of this user guide                                                                                                                                                                                                                                                                                                                                                                              |

Table 42. Core Revision History

| Core version | CORE_INFO | Changes                                                                                                                                                                                                                                                                                        |
|--------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 2.0          | 0x0200    | Removed the m_axi_resetrn output and the RESET_HOLD_CYCLES parameter. Added the RECOVERY register with UNBLOCK, and STATUS bits BLOCKED, WR/RD_RESP_BUSY and WR/RD_CMD_STUCK. Recovery became an explicit software sequence. Sources converted to SystemVerilog. <b>Breaking change.</b>       |
| 1.2          | 0x0102    | Fixed two AXI compliance defects on the control port: AWREADY / WREADY were asserted while a BVALID was outstanding, and ARREADY while an RVALID was, so a pipelining master lost responses. Fixed a rule-register byte-merge defect when ADDR_WIDTH < 32. Removed dead discard-pending logic. |
| 1.1          | 0x0101    | Split the single access-violation assertion into separate read and write assertions, which exposed that no test had ever denied a read.                                                                                                                                                        |
| 1.0          | 0x0100    | Initial release                                                                                                                                                                                                                                                                                |